

Pytalon 2.3 Architecture Documentation

M. Qasim Farooqi . Vexqyn

Abstract

We present the architecture of Pytalon 2.3, a non-AI, rule-based Python assistant implemented entirely in the Python standard library. The system separates a static knowledge base from a dynamic memory store, and couples parallel intent detection with sequential state refinement and confidence-gated answer validation. Linguistic matching is performed by *difflib.SequenceMatcher* (Ratcliff-Obershelp), a standard-library primitive; no third-party fuzzy-matching package is used. We describe the static phrase and keyword database, the custom natural-language pipeline, the validator decision procedure, the permanent and session memory subsystems, the behavior model, and the empty-input recovery path. The design rule set is explicit: ambiguous answers trigger clarification rather than prediction, and learner transcripts are never persisted.

1 Introduction

Conversational assistants for education are commonly implemented as wrappers over hosted language models. Such wrappers inherit model opacity, external runtime dependencies, and non-deterministic answer selection. Pytalon 2.3 takes the opposite design point. We construct a terminal assistant whose entire decision procedure is rule-based, threshold-based, and inspectable. The system teaches thirteen Python topics, validates learner code in a restricted executor, and maintains a permanent local profile of progress and communication style.

The architecture comprises four cooperating layers: an application layer that drives the teaching session; a body layer that interprets utterances through custom natural-language processing; a dynamic database that records session and long-term state; and a static knowledge base that enumerates phrases, topic keywords, and numeric thresholds. Answer understanding combines exact phrase sets, a deterministic grammar of negation and polarity, and continuous similarity scores produced by *SequenceMatcher*.

1.1 Contributions

Our main contribution includes:

Architecture. A layered separation of teaching, language understanding, persistence, and static knowledge, with single-responsibility module boundaries.

Custom language pipeline. Parallel intent scoring over seventeen detectors, followed by sequential refinement in a session state machine, with explicit confidence thresholds and negation calibration.

Dual-store memory. Session random-access state and permanent JSON progress, written atomically. The privacy invariant is that behavioral patterns and progress are retained while raw utterances are not.

Honest validation. A decision procedure that prefers clarification to guesswork on mixed polarity, ambiguous exits, and empty input, and that marks topic completion only after a practice execution succeeds.

1.2 Roadmap

In the remainder of this document, we first introduce the layered architecture and package layout (Section 2). We then describe the static knowledge database (Section 3) and the custom language pipeline (Section 4). Response validation and the matching engine are given in Sections 5 and 6. Dynamic memory and behavior modeling appear in Sections 7 and 8. Empty-input recovery is treated in Section 9, the practice executor in Section 10, and dependency constraints in Section 11. Section 12 concludes with limitations.

2 Architecture

We first introduce the module decomposition. Then we present the session state machine. For interface details not explicitly mentioned, call sites import body and memory symbols through thin compatibility facades (*validators.py*).

2.1 Layer Decomposition

```
APP learning.py | intro.py | topics_basic.py | topics_intermediate.py

Session flow, curriculum, practice orchestration

|

BODY pytalon_body/response_validators.py | intent_engine.py | behavior_learner.py

Parallel intent detection | answer validation | style counters

|

DYNAMIC Pytalon_Memory/conversation_context.py | memory_store.py | store/*.json

Session RAM | profile | progress | snapshots

+-----+-----+

STATIC ENGINE

config.py utils.py

phrases, keywords, SequenceMatcher scoring

thresholds, rules practice executor

+-----+-----+

STDLIB difflib | re | json | os | sys | io | datetime
```

Figure 1: Layer decomposition of Pytalon 2.3. Static knowledge is read-only. Dynamic state is written under the memory store. The body layer performs language understanding without disk I/O; the application layer performs teaching without NLP.

Teaching code does not own language processing. Memory code does not own dialogue. Intent code does not own persistence. That separation is the principal structural change relative to earlier releases.

2.2 Package Layout

The repository root contains the application entry point and curriculum modules. The *pytalon_body* package implements validation, intent detection, and behavior estimation. The *Pytalon_Memory* package implements session context and permanent storage under *store/*. Table 1 lists the public surface of each module.

Module	Responsibility
learning.py	Session lifecycle, teaching loop, memory boot and farewell
intro.py	Identity banner, topic catalog, opening dialogue
config.py	Static phrase, keyword, threshold, and rule database
utils.py	Matching engine, practice executor, topic menu
validators.py	Compatibility re-export of body validators
topics_basic.py / topics_intermediate.py	Curriculum topics 1-13 and deep-dive modules
response_validators.py	Yes / no / defer / exit / menu decision procedure

Module	Responsibility
intent_engine.py	Parallel intent scoring and fallback recovery
behavior_learner.py	Style counters and adaptive phrasing
conversation_context.py	Session state machine and style totals
memory_store.py	Profile, progress, snapshots, reports

Table 1: Module responsibilities.

2.3 Session State Machine

Let the session state be denoted by s in S with $S = \{\text{greeting, menu, topic, practice, done}\}$. Transitions are driven by validator outputs and teaching milestones. Table 2 gives the transition relation.

State s	Invariants	Permitted transitions
greeting	Identity issued; profile loaded	menu, topic, exit->done
menu	Topic catalog exposed	topic, exit->done
topic	One lesson active	practice, menu, topic, exit->done
practice	Sandboxed execution active	topic, exit->done
done	Progress flushed to disk	-

Table 2: Session states and transitions.

3 Static Knowledge Database

We now describe the static database. All linguistic priors of the system reside in *config.py* as immutable Python literals at runtime. No write path targets this module. Downstream components freeze list members into hash sets for exact matching.

3.1 Schema

The database is partitioned into eight groups. Answer banks supply phase-specific affirmative, negative, and exit phrases for general prompts, example prompts, and question prompts. Conversation patterns supply greeting, farewell, gratitude, confusion, uncertainty, defer, repeat, and clarification triggers. The topic graph maps each of the thirteen lesson names to a keyword multiset and a request-pattern list. Grammar sets encode exit first words, casual fillers, teach-request remainders, skip phrases, and mixed-polarity tokens. Matching hygiene sets list negation cues and filler words used for score calibration and keyword extraction. Memory language patterns enumerate learner utterances that request profile, export, path, or rename. Behavior markers and numeric thresholds define style classification. Practice fixtures supply canned *input()* values for the executor.

Group	Representative symbols	Consumer
Answer banks	YES_RESPONSES, NO_RESPONSES, EXIT_RESPONSES, phase variants	response_validators
Conversation	GREETING_PATTERNS, FAREWELL_PATTERNS, DEFER_PATTERNS, ...	intent_engine
Topic graph	TOPIC_KEYWORDS, TOPIC_REQUEST_PATTERNS, TOPIC_MATCH_THRESHOLD	intent_engine
Grammar	_TEACH_PREFIXES, _SKIP_PHRASES, _EXIT_FIRST_WORDS, _CASUAL_FILLERS	response_validators
Hygiene	NEGATION_WORDS, FILLER_WORDS, COMMON_SHORT_WORDS	utils, response_validators
Memory language	MEMORY_REQUEST_PATTERNS, MEMORY_EXPORT_PATTERNS, ...	intent_engine, memory_store
Behavior	BEHAVIOR_CASUAL_MARKERS, BEHAVIOR*_THRESHOLD	behavior_learner
Practice	PRACTICE_INPUT_RESPONSES	utils practice executor

Table 3: Static database groups.

3.2 Topic Graph

Let T be the set of topic names and $K(t)$ the keyword set of topic t . A learner phrase p is scored against each t in T by the matching function of Section 6. A topic is emitted only when the maximum score meets $\theta_{\text{topic}} = 0.65$. Generic single-word inputs in `COMMON_SINGLE_WORDS` are blocked from claiming a topic, and a bare mention of 'python' is insufficient unless a second keyword of that topic is present.

4 Custom Language Pipeline

We present the language pipeline in three stages: normalization, parallel intent detection, and sequential state refinement.

4.1 Normalization

Let u denote a raw utterance. We define the canonical form $v(u)$ by case folding, removal of punctuation except apostrophes, and collapse of whitespace. Command prefixes in `COMMAND_PREFIXES` (for example `/`, `!`, `#`) are stripped from the first token. Exact membership tests are then evaluated against prebuilt sets Y , N , E , D derived from the answer banks.

4.2 Parallel Intent Detection

detect_conversation_intent evaluates every scorer on every input in a single pass. Let $C(i)$ denote the confidence recorded for intent i . The selected intent is $\text{argmax}_i C(i)$ after priority suppression rules. Scorer order does not determine the winner.

```
u --> v(u) --> topic (smart + keyword) C in [0,1], gate theta_topic

--> beginner / help / practice C = 0.70-0.85

--> greeting / farewell / ... C = 0.90

--> yes / no / exit sets C = 0.95

--> question / defer / repeat C = 0.70-0.85

--> memory request / manage C = 0.90-0.95

|

v

priority rules --> argmax C(i)
```

Figure 2: Intent scoring. All detectors evaluate $v(u)$ in parallel. Confidence values are written to a score table; priority rules then suppress dominated intents; the maximum remaining confidence is selected.

Priority rules implement the following partial order. Substantive intents dominate social openers at equal or lower confidence. Conversation intents dominate a bare topic request. High-priority intents (gratitude, clarification, general question, defer) dominate an unnamed topic request. Memory intents are selected unless a named topic is present. Combined help signals raise $C(\text{help})$ to 0.95.

4.3 Sequential State Refinement

After detection, ConversationContext applies the winning intent to the current state s , the pending topic queue, and the dialogue history. Last-response recall reads the most recent assistant turn. Mid-prompt topic requests populate a pending-topic field that the teaching loop consumes after the active lesson returns. This stage is strictly sequential and conditions every subsequent decision on the refined state.

5 Response Validation

Validators convert free-text answers into the finite decision alphabet $\Delta = \{\text{yes, no, defer, exit}\} \cup T$, where T is the set of topic names. The procedure is staged: exact set membership, deterministic grammar, then continuous scoring. Algorithm 1 summarizes the general yes/no/defer/exit path.

Algorithm 1 Decision procedure for a prompt answer

1. Compute $v(u)$. If $v(u)$ in $Y \cup N \cup E \cup D$, return the corresponding symbol.
2. If the first token is an exit word and a non-filler remainder is present, request clarification and re-prompt.
3. If the line is a pure exit command with optional fillers, return exit.
4. If a skip phrase is present, return no.
5. If an affirmative multiword phrase is present and all extra tokens are casual fillers, return yes.
6. If the line is a teach-request with remainder in the allowed set and no other topic named, return yes.
7. If another topic is named, enter the topic-switch path.
8. If explicit yes and no tokens co-occur, request clarification and re-prompt.
9. Otherwise score $v(u)$ against Y, N, E, D with the matching function of Section 6 and the negation cap of Eq. (2).
10. Return the unique symbol whose score is at least $\theta_{\text{ans}} = 0.75$ and strictly maximal; else re-prompt.

5.1 Grammar Rules

Four grammars prevent systematic false positives. Negation grammar maps any cue in `NEGATION_WORDS` to a negative reading. Skip grammar elevates explicit skip phrases above casual affirmation. Teach grammar accepts 'teach me' and 'teach me this' as affirmation of the active topic while rejecting 'teach me ' as a switch. Exit grammar accepts fillers after an exit first word but rejects lines such as 'stop using print', which share an exit first word without denoting session termination. Mixed polarity is detected only from explicit yes and no tokens; scores alone never create a mixed label.

5.2 Thresholds by Phase

Function	Decision set	Threshold
<code>get_global_valid_input</code>	Δ general	$\theta_{\text{ans}} = 0.75$
<code>get_global_examples_valid_input</code>	yes / no	$\theta_{\text{ex}} = 0.65$
<code>get_global_user_question_valid_input</code>	yes / no / exit	$\theta_{\text{ans}} = 0.75$
<code>get_global_menu_choice</code>	topic index or exit	range + exit grammar
<code>get_global_question_content_input</code>	free text	$ u \geq 10$

Table 4: Validator phases.

Each finalized answer is observed once by the behavior engine through a decorator. The observation increments a counter only; the utterance is not stored.

6 Matching Engine

We define the similarity primitives used for topic resolution and answer scoring. Both operate on canonical strings and are implemented exclusively with *difflib.SequenceMatcher* and set algebra.

6.1 SequenceMatcher Primitive

SequenceMatcher implements Ratcliff-Obershelp gestalt matching. For sequences a and b it recursively extracts the longest contiguous matching block and recurses on the left and right residuals. The reported ratio is

$$R(a, b) = 2 M(a, b) / (|a| + |b|) \quad (1)$$

where $M(a, b)$ is the total length of matched blocks. R takes values in $[0, 1]$ and is deterministic. The class ships with CPython in *Lib/difflib.py*. It is a standard-library primitive and is not installed from the Python Package Index.

6.2 Relation to Third-Party Fuzzy Matching

Packages such as *fuzzywuzzy*, *rapidfuzz*, and *thefuzz* are external distributions that typically compute Levenshtein or related edit distances and expose scores on a 0-100 scale. They occupy *site-packages/* and require package installation. Pytalon does not import those packages. The distinction is categorical: *SequenceMatcher* is a language primitive; the aforementioned libraries are products. Table 5 records the contract.

Property	<code>difflib.SequenceMatcher</code>	<code>fuzzywuzzy</code> / <code>rapidfuzz</code> / <code>thefuzz</code>
Distribution	CPython standard library	Third-party package
Installation	None	pip install
Algorithm	Ratcliff-Obershelp	Edit-distance family
Output	R in $[0, 1]$	Typically 0-100
Used in Pytalon	<code>utils.py</code> only	Not imported

Table 5: Matching dependency contract.

6.3 Composite Scorers

`smart_detection(s1, s2)` returns containment 1.0 on padded phrase inclusion or word-set subset, otherwise the maximum of $R(s1, s2)$, $R(w, s2)$ over words w of $s1$, the symmetric word scores, and a reversed-order score. `smart_validators(s1, s2, w1, w2)` first computes keyword overlap after filler removal. Overlap of at least two shared keywords across both string and keyword pairs yields 1.0; overlap of one yields 0.5. Otherwise the *SequenceMatcher* ladder is applied and the negation cap is enforced:

$$S_{cap} = \min(S, 0.5) \text{ if } \text{neg}(s2) \text{ and not } \text{neg}(s1), \text{ else } S \quad (2)$$

where $\text{neg}(\cdot)$ denotes presence of a cue in `NEGATION_WORDS`. Equation (2) is the calibration stage: it prevents a positive score from exceeding the decision threshold when the reference phrase is negative and the learner utterance is not.

7 Dynamic Memory

We now describe persistent state. The dynamic database has two tiers: a session tier held in process memory and a permanent tier serialized as JSON under *Pytalon_Memory/store/*. The privacy invariant is that progress and derived counters are retained while raw utterances are discarded.

7.1 Session Tier

ConversationContext records a role/content history, the set of topics taught in the session with multiplicities, style counters, the current state *s*, and a pending-topic slot. Style counters include message count, character count, and counts of messages exhibiting capitalization, emoji, interrogation, and casual markers. The method *observe_input* updates counters only.

7.2 Permanent Tier

learner_profile.json stores name, optional username, and display name. *long_term_memory.json* stores completed topics, per-topic practice attempts, defer count, aggregate behavior totals, and a bounded session log (the most recent fifty rows). Dated snapshots under *session_snapshots/* archive profile and progress for offline inspection. *memory_report.txt* is written only on explicit learner request.

```
remember_*(args)

|
v

load_long_term() / load_profile() <-- cache

|
v

field update (topics, practice, prefs, behavior)

|
v

_write_json: write *.tmp --> os.replace(tmp, target)

|

+--> cache refresh
```

Figure 3: Write path. Mutations load through an in-RAM cache, update fields, and commit with an atomic replace. Readers may observe either the previous or the next complete document, never a partial file.

7.3 Lifecycle

On boot, *ensure_learner_profile* obtains or creates the profile, *build_welcome_summary* emits completed-topic statistics and a suggested next topic, and *seed_context_from_memory* copies completed topics into session state. On termination of any kind-including *interrupt-record_session_end* increments session counters, merges behavior totals, truncates the session log if necessary, and writes a snapshot. All optional memory operations are guarded so that storage failure cannot abort a lesson.

8 Behavior Model

The behavior model estimates communication style from derived features only. For an utterance *u* we extract the feature vector

$$\phi(u) = (\text{len}(u), \text{caps}(u), \text{emoji}(u), \text{query}(u), \text{casual}(u)) \quad (3)$$

where each coordinate is Boolean except length, and casual(u) is true when the tokenized utterance intersects BEHAVIOR_CASUAL_MARKERS. Session aggregates form ratios r_casual and r_emoji over the message count.

8.1 Trait Rules

Traits are assigned by threshold rules with published constants. Voice is casual when message count is at least three and r_casual ≥ 0.30 or r_emoji ≥ 0.25 ; otherwise voice is steady. Offer affinity holds when at least four polarity answers exist and the affirmative rate is at least 0.60. Gentle pacing holds when the persistent defer count is at least two, or when at least six polarity answers exist with a negative majority. A topic is marked revisited when its teach multiplicity reaches two.

Trait	Rule
voice = casual	msgs ≥ 3 and (r_casual ≥ 0.30 or r_emoji ≥ 0.25)
likes_offers	answers ≥ 4 and yes-rate ≥ 0.60
gentle_pacing	defer_count ≥ 2 or (answers ≥ 6 and no > yes)
revisited_topics	teach count ≥ 2

Table 6: Behavior trait rules.

Traits modulate surface phrasing only: opening register, softening of the teach prompt, and ordering of suggested topics. They do not alter acceptance thresholds of the validator.

9 Empty and Unrecognized Input

We treat degraded input as a first-class case. Empty utterances and utterances that survive no scorer must not be silently dropped and must not force a guess. The recovery ladder is deterministic and always produces either a recovery reply or a menu fallback.

```
u = empty -> handle_empty_input -> intent empty

retry_count += 1; after 2, expose topic menu

|u| > 10000 -> handle_unrecognized_input (length bound)

no scorer hit -> handle_unrecognized_input

1. known Y/N/E/D: never labeled unrecognized

2. greeting / gratitude / farewell / help patterns

3. near-topic if max score >= theta_topic

4. short (<10) vs general help text

all paths -> observe_input(u) updates style counters only
```

Figure 4: Recovery ladder. Empty input increments a retry counter in the opening loop. Unrecognized input is re-checked against known answer sets and the static conversation patterns before a near-topic suggestion or a short/general help reply is emitted. Style counters are updated on every path.

Conversational learning is preserved under recovery. Failed turns still contribute length, capitalization, and emoji evidence to the behavior model, so subsequent register choices remain calibrated. The opening loop converts repeated unrecognized turns into a menu presentation rather than an unbounded retry loop.

10 Practice Execution

The practice executor runs learner code under a restricted namespace. Guards are static and dynamic. Static analysis rejects enumerated infinite-loop patterns before execution. Dynamic tracing installs a line-event hook with step budget $N_{\text{step}} = 200\,000$; exceeding the budget aborts execution with a diagnostic. Standard output is captured. Builtins exit and quit are replaced by raising stubs. Calls to input are served from the canned fixture list so the interactive terminal is never consumed. Keyword requirements are matched with operator-aware token boundaries so that $>$ does not satisfy $\geq$. Per-topic check functions add structural constraints. A topic is marked complete in permanent memory only when execution and all checks succeed.

Guard	Mechanism
Runaway loops (static)	Pattern pre-check on the source text
Runaway loops (dynamic)	sys.settrace line budget N_{step}
Terminal theft	input served from PRACTICE_INPUT_RESPONSES
Process exit	exit / quit rebound to raise
Keyword false positives	token-boundary operator matching
Completion	persist only on full success

Table 7: Practice executor guards.

11 Dependency Constraints

The implementation depends only on the Python standard library. Table 8 enumerates every non-built-in import in the product modules. There is no requirements manifest declaring third-party distributions, and no module imports a package from *site-packages/*.

Import	Role
difflib	SequenceMatcher ratio $R(a, b)$
re	beginner and question regular expressions; token splits
io	StringIO capture in the practice executor
sys	stdout rebinding, settrace, process exit
json	profile, progress, and snapshot documents
os	path resolution and atomic replace
datetime	timestamps and session duration

Table 8: Standard-library import inventory.

11.1 Honesty Invariants

The decision layer maintains the following invariants. (i) The system does not claim to be a learned model. (ii) Mixed polarity and ambiguous exits elicit clarification rather than a forced choice. (iii) Permanent memory is never deleted automatically. (iv) Raw utterances are never written to the store. (v) Topic completion requires a successful practice execution. (vi) Memory subsystem failure does not terminate a lesson.

12 Conclusion and Limitations

We have presented the architecture of Pytalon 2.3. The system demonstrates that a non-AI assistant can expose a complete, inspectable decision procedure: a static knowledge database, a custom language pipeline with parallel detection and sequential state refinement, confidence-gated validation calibrated by negation rules, and a dual-store memory that retains patterns and progress without transcripts. Similarity is computed with the standard-library Ratcliff-Obershelp primitive rather than a third-party fuzzy-matching distribution.

Limitations follow from the rule-based design. Coverage is bounded by the static phrase and keyword sets; utterances outside those sets fall to the recovery ladder. Thresholds are fixed constants rather than fitted parameters. Topic resolution is lexical and does not model paraphrase beyond SequenceMatcher similarity. The practice executor is not a security sandbox against hostile code; it is a teaching guard for well-formed exercises. Future work includes threshold calibration from session statistics, expansion of the topic graph, and structured schema for snapshot review.

Document control. Title: Pytalon 2.3 Architecture Documentation. Version: 2.3. Author: M. Qasim Farooqi (Vexqyn). License: MIT. Scope: technical architecture of the non-AI Python assistant. Matching contract: difflib.SequenceMatcher; no fuzzy-matching package dependency.